OPCVM GSM

SPÉCIFICATIONS DE RÉFÉRENCE ARCHITECTURALE

Normes de conception, sécurité, API, workflows, déploiement et critères d'acceptation

Objectif - Fournir une référence stable pour faire évoluer OPCVM GSM sans casser les principes multi-SGI, l'intégrité financière, la traçabilité des décisions et la séparation entre interface, logique métier et persistance.

Référentiel d'architecture
Version 2.0 - 10 novembre 2026

État de référence de la maquette et du backend au 10 novembre 2026

1. Portée du référentiel

- Ce document s'applique au frontend React/Vite, au backend Django/DRF, aux bases PostgreSQL, au cache/queue Redis, aux tâches Celery et aux stockages documentaires.
- Il couvre Gestion sous mandat, Gestion libre, CRM, Marchés, Ordres, Money Management, Cession_Retrait, Comité de gestion, Administration SGI et Documentation.
- Toute nouvelle fonctionnalité doit préciser son propriétaire métier, sa source de vérité, son tenant, ses permissions, ses règles de validation, ses effets sur le ledger et son audit.

2. Principes architecturaux obligatoires

ID	Principe	Exigence
ARC-01	Source de vérité unique	Aucune donnée métier critique ne doit être maintenue indépendamment dans plusieurs stockages.
ARC-02	API-first	Le frontend ne modifie jamais directement PostgreSQL ; toute mutation passe par une API métier.
ARC-03	Tenant explicite	Tout objet appartenant à une SGI porte institution_id ou une relation démontrable vers le tenant.
ARC-04	Validation serveur	Les règles financières, permissions et contraintes sont vérifiées côté backend même si l’UI les applique déjà.
ARC-05	Atomicité	Une opération financière multi-écritures réussit entièrement ou est annulée entièrement.
ARC-06	Traçabilité	Les changements de statut, décisions, exécutions et actions sensibles laissent un historique.
ARC-07	Séparation marché / métier	Les données de marché ne sont pas confondues avec les données clients/portefeuilles.
ARC-08	Compatibilité progressive	Les bridges temporaires sont autorisés mais doivent avoir une trajectoire de suppression.

3. Architecture applicative de référence

Zone	Composants	Règle
Présentation	React/Vite, composants, charts, exports	Pas de règle métier critique uniquement côté navigateur.
API	DRF serializers/viewsets/services	Contrôles d’entrée, permissions, orchestration.
Domaine	Services métier Django	Transactions, états, invariants et workflows.
Persistance	ORM Django / PostgreSQL	Source de vérité durable.

Zone	Composants	Règle
Asynchrone	Celery / Redis	Traitements longs, notifications, recalculs.
Documents	S3/MinIO/Blob ou static versionné	Fichiers lourds et pièces versionnées.

4. Convention multi-SGI

- Le tenant courant est résolu une seule fois par requête et réutilisé par les viewsets/services.
- Un utilisateur sans membership actif pour l'institution demandée reçoit un refus 403.
- Une clé étrangère soumise par le client doit être revalidée dans le même tenant.
- Les agrégations et exports doivent être tenant-scoped avant tout calcul.
- Les tâches asynchrones doivent transporter explicitement institution_id et revalider le contexte à l'exécution.

5. Rôles et habilitations

Rôle	Responsabilités de référence
ADMIN_SGI	Administration métier de la SGI, utilisateurs habilités, supervision.
HEAD_PM	Supervision de la gestion, comité, décisions et contrôles.
PORTFOLIO_MANAGER	Portefeuilles attribués, décisions, ordres, liquidité, CRM utile.
CUSTOMER_OFFICER	Relation client, CRM, demandes, réception/qualification des intentions.
TRADER	Négociation et routage/exécution des ordres.
BACK_OFFICE	Règlement, rapprochements, avis, mouvements et contrôles post-trade.
COMPLIANCE	KYC, contraintes, conformité et alertes.
AUDITOR	Lecture et audit sans mutation métier.
CLIENT	Accès uniquement à son espace et à ses données autorisées.

6. Authentification et Administration SGI

1 Authentification API de l'utilisateur et sélection de l'institution autorisée.

↓

2 Depuis Gestion sous mandat, l'utilisateur demande Administration SGI.

↓

3 Le backend vérifie le code d'attribution hashé, le rôle, l'état du membership et admin_access_enabled.

↓

4 Un jeton signé, court et à usage limité ouvre /admin/bridge/.

↓

5 Django crée la session Admin ; l'utilisateur reste is_staff et non superuser sauf administrateur technique.

7. Convention API REST

Règle	Spécification
Version	Préfixe /api/v1/ pour les ressources publiques du backend applicatif.
Format	JSON UTF-8 ; dates ISO 8601 ; montants décimaux, jamais float financier côté base.
Auth	Bearer JWT ; institution explicite dans X-Institution-ID tant que le contexte n’est pas porté par le token.
Pagination	Obligatoire pour listes volumineuses ; tri et filtres contrôlés.
Erreurs	400 validation, 401 non authentifié, 403 non autorisé, 404 objet absent du tenant, 409 conflit métier.
Idempotency	Clé d’idempotence recommandée/obligatoire pour imports, paiements, ordres externes et callbacks.
Audit	Correlation/request ID propagé dans logs et événements sensibles.

8. Standards de données financières

- Montants et quantités : Decimal avec précision explicite en base.
- Devise : code ISO ou code interne stable ; la devise source est toujours conservée.
- Prix / cours : conserver source, date/heure et convention ; éviter de reconstruire une donnée de marché officielle absente.
- Positions : dérivées des mouvements/lots ; les snapshots sont des vues historisées, pas une saisie libre.
- CMP/PRU : recalcul déterministe depuis les lots/mouvements selon convention validée.
- Performance : distinguer valeur de marché, flux externes et méthode de rendement (TWR/MWR).

9. Workflow de référence - ordre

1 Création du brouillon depuis ticket, chat, ordre groupé, comité ou API.

↓

2 Contrôle portefeuille, mandat, KYC/conformité, cash ou titres disponibles.

↓

3 Approbation selon rôle et routage au négociateur.

↓

4 Réception des exécutions partielles/totales.

↓

5	Allocation et écritures ledger atomiques.
↓	
6	Mise à jour positions, cash, valorisations, avis et historique de statut.

10. Workflow de référence - retrait et cession interne

1	Création WithdrawalRequest avec montant, devise et portefeuille.
↓	
2	Validation des limites et calcul du besoin de trésorerie.
↓	
3	Plan de cession par instruments et quantités disponibles.
↓	
4	Recherche de contreparties internes admissibles.
↓	
5	Transfert interne ou vente marché pour le reliquat.
↓	
6	Disponibilité cash après règlement, puis paiement du retrait.

11. Workflow de référence - comité

1	Ouverture CommitteeSession, quorum et ordre du jour.
↓	
2	Création CommitteeDecision avec instrument, sens, allocation, type d’ordre et justification.
↓	
3	Création DecisionTarget par portefeuille éligible.
↓	
4	Génération/rattachement des ordres et enregistrement DecisionExecution.
↓	
5	Calcul du résultat et création DecisionDeviation pour chaque écart significatif.
↓	
6	Restitution au comité suivant avec résultats, alignement et motifs d’écart.

12. Workflow de référence - CRM

1. Création/qualification d’un Prospect ou sélection d’un Client existant.
2. Enregistrement des activités et tâches de suivi.
3. Création d’une opportunité avec montant, probabilité, étape et responsable.
4. Gestion des demandes client et échéances.
5. Conversion Prospect -> Client via service backend contrôlé.
6. Création éventuelle d’un mandat/portefeuille sans dupliquer le référentiel client.

13. Import et intégration externe

- Tout fichier passe par staging avant commit.
- Le hash du fichier, l'utilisateur, l'institution, le timestamp et le résultat du contrôle sont conservés.
- Le mapping doit distinguer données source, normalisées et erreurs de validation.
- Le commit doit être rejouable ou protégé contre les doublons.
- Les APIs externes utilisent le même principe d'inbox/staging pour les flux critiques.

14. Sécurité de production

Contrôle	Exigence minimale
Transport	HTTPS partout ; HSTS après validation du domaine.
Secrets	Variables serveur / secret manager ; jamais VITE_* pour mot de passe, clé privée ou secret.
Cookies	Secure, HttpOnly lorsque applicable, SameSite adapté.
CSRF/CORS	Listes de domaines explicites ; pas de wildcard en production.
MFA	Recommandé pour utilisateurs internes habilités et obligatoire pour profils sensibles à terme.
Rate limiting	Endpoints auth, admin-access, import et actions sensibles.
Fichiers	Contrôle MIME, taille, antivirus si disponible, stockage non exécutable.
Audit	Historique des actions sensibles, accès admin, changements de rôles et exports confidentiels.

15. Observabilité et exploitation

- Logs structurés avec request_id, institution_id, user_id, endpoint, durée et résultat.
- Métriques : taux d'erreur, latence API, jobs Celery, imports, ordres, temps de traitement des retraits.
- Alerting sur indisponibilité DB/Redis, échecs Celery, erreurs d'import répétées et anomalies de sécurité.
- Sauvegardes PostgreSQL testées par restauration ; politique de rétention documentée.

16. Déploiement et environnements

Environnement	Règle
Local	Django 8001, Vite 5173, Docker PostgreSQL/Redis si disponible, fallback SQLite uniquement pour test rapide.
Intégration	Base dédiée, migrations automatiques contrôlées, données de test non sensibles.
Préproduction	Topologie proche production, HTTPS, tests de migration et sécurité.
Production	PostgreSQL managé ou durci, Redis, reverse proxy, static, sauvegardes, observabilité et secrets gérés.

17. Tests et critères d’acceptation

Famille	Critères
Unitaires	Règles métier, services, conversions, validations et calculs.
API	Auth, tenancy, permissions, sérialisation, filtres, erreurs et idempotence.
Intégration	Ordre -> exécution -> ledger ; import -> staging -> commit ; comité -> ordre -> résultat ; CRM -> client.
Sécurité	Cross-tenant, élévation de privilège, CSRF/CORS, brute force admin-access, uploads.
Frontend	Parcours critiques, états vides/erreurs, responsive, téléchargements documentation.
Régression	Chaque modification conserve les parcours validés et le thème/ergonomie du produit.

18. Critères de production-ready

1. Aucune mutation critique dépend uniquement de localStorage ou d'une constante JavaScript.
2. Tous les endpoints métier sensibles ont des permissions tenant + rôle + objet.
3. Les imports disposent d'un commit atomique, d'une réconciliation et d'un rapport d'audit.
4. Le ledger produit positions, cash et valorisations cohérents et testés.
5. Les données de marché affichent source, date/heure et statut réel/simulé/différé/EOD.
6. La configuration domaine/HTTPS/CSRF/CORS est validée et les secrets ne sont pas exposés dans le bundle frontend.
7. Les sauvegardes, restaurations, logs, métriques et alertes d'exploitation sont testés.
8. Les workflows Cession_Retrait, Ordres, Comité et CRM ont des tests d'intégration complets.

19. Décisions d’architecture à conserver

Référence
Modular monolith d'abord, séparation claire des domaines, PostgreSQL source de vérité, institution_id partout où nécessaire, données de marché séparées, staging avant import, API métier entre frontend et base, et progression par modules réellement persistants avant microservices.